DATA ANALYST INTERNSHIP

Learning and Progress Report

Ashwina Sedhain

Introduction

This report documents my learning journey throughout the Data Engineer internship curriculum. The syllabus covered a wide range of topics from basic Python programming all the way to building real-time big data pipelines and deploying applications on Kubernetes. Over the course of this program, I worked on practical tasks for every topic, and this report reflects what I learned, what I built, and how my problem-solving ability grew through the process.

When I started, I had some basic knowledge of Python but had never worked with distributed systems, databases like Elasticsearch or Neo4j, or tools like Docker, Airflow, and Kafka. By the end, I had built a full capstone project that combined all of these technologies into one working system. That shift from knowing individual tools to connecting them together is the most important thing this internship gave me.

Week 1: Python Programming

The first week was about building a strong foundation in Python. I already knew the basics, but this week pushed me into areas I had not explored before, like decorators, async programming, and type annotations.

For the tasks, I started with a student grade calculator that used every type of Python operator, arithmetic, comparison, logical, bitwise, membership, and identity. This helped me understand not just what each operator does, but when to actually use them. I then built a student management system using loops and control structures, which was a simple CLI app but forced me to think about program flow carefully.

The module task was a library system where I created a Book class in one file and a Library class in another, then imported them into a main file. This was my first real experience with how Python modules and classes work together in a project structure rather than a single script.

The decorator tasks were the most challenging part of the week. I implemented three versions: basic chaining, parameterised decorators, and a combination of both. Understanding that a decorator is just a function that wraps another function took some time to click, but once it did, I could see how frameworks like FastAPI use them everywhere.

For concurrency, I wrote three separate programs to compare synchronous execution, multithreading, and multiprocessing. The synchronous version took 6 seconds to complete three tasks; the async version finished in 3 seconds. Seeing that difference in actual output made the concept real for me. I also learned when to use threading versus multiprocessing, threading for I/O-bound work, multiprocessing for CPU-bound work.

The MyPy task introduced me to type annotations. I wrote a function with proper type hints and intentionally introduced a type error to see how MyPy catches it before the code even runs. This was a small task but it changed how I write Python going forward, I now add type hints by habit.

Week 2: Databases

The database week was the most varied week of the curriculum because it covered four completely different types of databases. Each one has a different way of thinking about data, and switching between them in the same week was a good challenge.

PostgreSQL (SQL)

I used PostgreSQL with pgAdmin to build an internship syllabus tracker. The database had tables for modules, topics, status fields, and timestamps. I wrote queries to track which topics were completed, in progress, or pending, a practical use case that I actually used during the internship to monitor my own progress.

SQLAlchemy (ORM)

I built a personal expense tracker. This was more complex than raw SQL because I had to define ORM models for Category and Expense, set up relationships between them, and write a CLI application that let users add categories, add expenses under those categories, and view all expenses. Working with SQLAlchemy taught me how to think about database design in Python terms rather than SQL terms.

MongoDB (Document Database)

I built a quiz game where questions, options, answers, and hints were stored as documents in a MongoDB collection. The game shuffled questions randomly, let users ask for hints, and showed a score at the end with star ratings. This task showed me how document databases are much more flexible than relational ones when data has varying structure.

Elasticsearch (Search Engine)

I built a Nepali recipe finder with a custom index mapping that defined how each field should be analysed, recipe names as full text, ingredients as keywords, ratings as floats. A search function filtered by cuisine, difficulty, minimum rating, and maximum prep time simultaneously. This was my first experience with a search engine, and it was impressive how fast and flexible the queries were.

Neo4j (Graph Database)

I built a social network graph with people, skills, and companies as nodes, and relationships like FRIEND, HAS_SKILL, and WORKS_AT between them. Cypher queries found friends of friends, suggested new connections, and found people who share skills. Graph databases think about data in a completely different way from tables or documents, and this task made that difference very clear.

Week 3: Data Science

The data science week covered statistics, machine learning, and data visualisation. I worked entirely in Jupyter notebooks, which was a good environment for exploring data step by step.

I started with data preprocessing using a messy HR dataset. The raw data had missing values, inconsistent formats, duplicate rows, and outliers. I cleaned it step by step, filling missing values with medians and modes, standardising text fields, removing duplicates, and capping outliers using the IQR method. This preprocessing work takes up most of the time in any real data project, and doing it on a genuinely messy dataset taught me more than any clean example would have.

Machine Learning

I worked on three types of problems:

- Regression: Used the Boston Housing dataset to predict house prices with Linear Regression, Ridge, and Lasso, learning how regularisation helps when features are correlated.
- Classification: Used the Titanic dataset to predict survival with Logistic Regression, Random Forest, and SVM, learning about train-test splits, confusion matrices, and why accuracy alone is insufficient with imbalanced classes.
- Clustering: Used the Iris dataset with K-Means and visualised clusters using PCA to reduce dimensions to two.

Data Visualisation

I created bar plots, scatter plots, histograms, box plots, and pie charts using Matplotlib. I also built a Dash application using the WFP food prices dataset, my first interactive web dashboard in Python, with dropdowns and charts that updated based on user selection.

Mixpanel Analytics

I integrated Mixpanel analytics into a project management application, tracking user events like sign-ups, workspace creation, task completion, and page views. The analysis revealed that users who create a workspace have 80% retention at day 7 compared to 15% for users who do not, and that 60% of users never discover the workspace creation button. These findings were presented as an executive dashboard with retention heatmaps and funnel visualisations.

CI/CD: Git, Testing, and GitHub Actions

The CI/CD section was about professional software development practices rather than a specific technology. I built a Python calculator library with eight operations and used it as the base for demonstrating the full development workflow.

I initialised a Git repository and made over fifteen commits following the conventional commit message format using prefixes like feat, fix, test, ci, and docs. I created feature branches for each new set of operations, merged them into a develop branch, and then merged develop into main. I demonstrated both fast-forward and non-fast-forward merges, and intentionally created a merge conflict to practise resolving it manually.

I wrote thirty unit tests using pytest covering normal cases, edge cases like zero inputs, and failure cases like division by zero, plus five integration tests that chained multiple operations together. I introduced a bug intentionally, making the divide function return multiplication, and then fixed it, demonstrating the full QA cycle.

The GitHub Actions pipeline runs automatically on every push. It runs flake8 to check code style, then runs unit and integration tests, and finally generates a coverage report. Having the pipeline catch issues automatically before code is merged is something I now consider essential for any project.

Week 4: Big Data Volume, Hadoop Ecosystem

The big data week was the most technically demanding week of the curriculum. I worked with the Brazilian e-commerce dataset from Kaggle, which had nine CSV files covering orders, customers, products, sellers, and payments.

HDFS and MapReduce

I started by loading the data into HDFS using shell scripts. Seeing data distributed across the Hadoop filesystem was the first time I understood what distributed storage actually means in practice. I then wrote a MapReduce job from scratch, a mapper that emitted product IDs and sale amounts, and a reducer that aggregated total sales per product.

Hive

I wrote eight analytical SQL queries: top product categories by revenue, monthly sales trends, top customers by spending, state-wise analysis, delivery performance, and order status distribution. Hive bridged the familiar SQL world and the distributed storage world.

Apache Spark

I built a full ETL pipeline that loaded the CSV files, performed data quality checks, joined four datasets, created derived columns like total order value and delivery days, and saved results as Parquet files. The Spark MLlib task built a customer churn prediction model using Logistic Regression and Random Forest, with feature engineering, train-test splitting, and evaluation using accuracy and AUC metrics.

HBase

I built a data manager class using HappyBase that created tables with column families, loaded customer and product data from CSV files, and demonstrated all CRUD operations, create, read, update, and delete, along with scan operations filtered by state and category.

Big Data Speed: Kafka and Spark Streaming

This section covered real-time data processing, a completely different challenge from batch processing. I implemented a simplified Lambda Architecture with three layers.

The producer script simulated twenty users generating login, click, and purchase events continuously with weighted probabilities matching real user behaviour (50% clicks, 30% logins, 20% purchases). Events were sent to a Kafka topic with three partitions, allowing parallel consumption.

The Spark Structured Streaming job read from Kafka in real time and ran two queries simultaneously: one grouping events into ten-second windows counting them per user and event type, and another filtering only purchase events for instant display. The windowed aggregation used watermarking to handle late-arriving events correctly.

The batch job read all historical events from Kafka at once and computed three aggregations: total activity per user, event type distribution, and most visited pages. Over a fifteen-minute run the pipeline processed 2,712 events and the distribution matched the weighted probabilities almost exactly, confirming the pipeline was working correctly.

Understanding the difference between the speed layer and the batch layer, and why you need both, was the key insight from this section. Real-time processing is fast but approximate; batch processing is slow but complete. Lambda Architecture runs both in parallel and combines the results.

APIs: Flask, FastAPI, and REST

The API section started with a direct comparison between Flask and FastAPI. I built the same simple user API in both frameworks. The comparison made it clear that FastAPI is faster to write, automatically validates request data through Pydantic, and generates interactive documentation without any extra work.

I then built the Smart Task Manager API, a complete, production-style FastAPI application with full CRUD operations for tasks, filtering by status and priority, pagination, automatic timestamps, and token-based authentication. The API returned proper HTTP status codes: 400 for validation errors, 401 for authentication failures, 404 for missing resources, and 500 for unexpected errors. The interactive Swagger UI at /docs made testing straightforward.

Building this API taught me how to structure a real FastAPI project across multiple files, separating the entry point, database setup, models, authentication, and routes into their own modules.

Docker: Containerisation

The Docker section changed how I think about running software. Before this, I would install dependencies directly on my machine and hope everything worked the same on another machine. Docker solves that problem completely.

I built a Todo API using FastAPI and containerised it with a Dockerfile. I then wrote a docker-compose file that ran two containers together, the API and a PostgreSQL database. The containers communicated using the service name defined in docker-compose, with database data stored in a named volume so it survived container restarts.

I pushed the image to Docker Hub, which meant anyone could pull and run the application with a single command without installing Python or PostgreSQL. The concepts of images, containers, volumes, networks, and docker-compose all became concrete through building and running this project.

Apache Airflow: Workflow Orchestration

The Airflow section was about automating and scheduling data pipelines. I built a five-task weather data pipeline that ran on a daily schedule.

The pipeline fetched live weather data from the Open-Meteo API for New York City, processed it to calculate minimum, maximum, and average temperatures, saved the processed record to a local data store, generated a human-readable report, and sent a success notification using a BashOperator. Tasks passed data to each other using XCom, Airflow's mechanism for sharing small pieces of data between tasks.

The pipeline had retry logic (three retries with exponential backoff) and proper error handling at each step. Monitoring the pipeline through the Airflow web UI and watching each task turn green one by one made the concept of a DAG very tangible.

Kubernetes: Container Orchestration

Kubernetes was the final and most advanced deployment topic. I deployed a FastAPI application to a local Kubernetes cluster using Minikube and worked through the core concepts one by one.

I wrote a Deployment manifest that ran two replicas of the application and configured liveness and readiness probes so Kubernetes could check whether each container was healthy. I wrote a Service manifest that exposed the application on a NodePort, and used a ConfigMap to store the application's message and version as environment variables, keeping configuration separate from the container image.

I demonstrated scaling by increasing the replica count from two to four with a single command and watching the new pods appear. I demonstrated rolling updates by building a v2 image, applying an updated ConfigMap, and watching Kubernetes replace the old pods one by one without any downtime. I demonstrated self-healing by deleting a running pod and watching Kubernetes immediately start a replacement.

Project 1: Real-Time News Analytics and Intelligent Data Pipeline System

---

This project is a complete real-time news analytics system. It automatically collects news articles from two sources every 30 minutes, processes them using machine learning to understand their meaning and sentiment, stores them in databases, and displays everything on a live dashboard that updates itself automatically. The entire system runs inside Docker containers so it can be started with a single command.

The goal of this project was to demonstrate how a modern big data pipeline works from start to finish, covering data collection, message streaming, data processing, machine learning, REST APIs, live dashboards, workflow automation, containerisation, cloud deployment, and automated testing all in one place.

Data Sources

The system collects news from two sources. The first is NewsAPI (newsapi.org), a service that provides access to articles from thousands of newspapers and websites through a simple HTTP API. Two endpoints are used: the top-headlines endpoint for the most popular current technology stories from the United States, and the everything endpoint for keyword searches across topics like technology, AI, and startups. Both return structured JSON so no HTML parsing is needed.

The second source is Hacker News (news.ycombinator.com), a popular technology news community run by Y Combinator. The official Hacker News Firebase REST API is used rather than scraping HTML directly. The system first fetches the list of top story IDs, then downloads each story's details in parallel using multiple threads so collection finishes quickly. Both sources return data in different formats, so everything is normalised into the same standard article shape so the rest of the pipeline does not need to know which source an article came from.

How the Pipeline Works

Every 30 minutes, Airflow triggers the collection pipeline automatically. The collector fetches articles from both NewsAPI and Hacker News, removes any duplicates it has already seen, and stores the raw articles in MongoDB. The ML pipeline then runs on each article to add sentiment scores and keywords, and the enriched articles are stored in PostgreSQL. At the same time, articles are published to a Kafka topic so the Spark streaming job can pick them up. The FastAPI backend reads from PostgreSQL and MongoDB to serve data to the dashboard, which refreshes every 30 seconds so users always see the latest information.

Codebase Structure

The scraper folder handles all data collection. The newsapi_client.py file connects to NewsAPI, hackernews_scraper.py fetches from the Firebase API, deduplicator.py tracks which articles have already been seen, and collector.py combines all three into one collect method that the rest of the system calls.

The kafka folder manages message streaming. The producer.py file publishes articles to Kafka topics as JSON messages, consumer.py reads messages and passes them to a handler,

and topics.py creates the three Kafka topics the pipeline uses: raw-news for incoming articles, processed-news for enriched articles, and error-events for failed messages.

The spark folder contains the Spark Structured Streaming job. The stream_processor.py file reads from the raw-news Kafka topic, cleans article text by removing HTML tags and extra whitespace, adds a processing timestamp, and writes results back to Kafka and to PostgreSQL as a continuous stream.

Machine Learning Pipeline

The ml folder contains four components combined into a single pipeline class. The sentiment.py file uses the VADER lexicon from NLTK to label each article as positive, negative, or neutral. The keywords.py file extracts the most important words from each article by removing stop words and counting remaining word frequencies. The clustering.py file groups similar articles together using K-Means on TF-IDF vectors. The trends.py file detects which keywords are growing in popularity by comparing counts across two rolling time windows. The pipeline.py file combines all four so the rest of the system only needs to call one method to get fully enriched articles.

API and Dashboard

The FastAPI backend is organised into four routers. The news router handles /news/latest and /news/search endpoints. The analytics router handles /analytics/sentiment, /analytics/trends, and /analytics/sources for chart data. The scrape router handles /scrape/run for on-demand collection and /scrape/debug for troubleshooting. The metrics router handles /metrics/live for system health statistics.

The Dash dashboard shows three metric cards at the top for total articles, articles today, and API uptime. Below that is a sentiment pie chart, a trending keywords bar chart, and a source distribution chart. At the bottom is a live news feed table where positive articles are highlighted green and negative articles are highlighted red. Everything refreshes automatically every 30 seconds.

Workflow Automation

The Airflow DAG that runs every 30 minutes has three tasks in sequence: collect_news fetches from both sources and stores in MongoDB, run_ml_pipeline loads the articles and runs sentiment and keyword analysis and stores in PostgreSQL, and publish_to_kafka sends the articles to the Kafka topic. A second cleanup DAG runs every night at 2am and deletes articles older than 30 days from both PostgreSQL and MongoDB to keep the databases lean.

Deployment and Testing

The system uses separate Dockerfiles for each service: the FastAPI service, Dash dashboard, Spark streaming job, and Airflow scheduler. The Kubernetes folder contains deployment manifests for every service including a HorizontalPodAutoscaler that automatically scales the API pods up to 6 replicas when CPU usage exceeds 70 percent.

The test suite covers unit tests for the scraper normalisation logic, deduplicator filtering, all ML components, and all FastAPI endpoints using a test client with mocked database calls. Integration tests verify the full data flow from collection through ML to database storage. The GitHub Actions CI/CD pipeline runs on every push to main, runs flake8 and pytest, builds all four Docker images if tests pass, pushes them to DockerHub, and deploys the updated images to the Kubernetes cluster.

Technologies Used

- Python 3.11 across all services
- FastAPI for the REST API with automatic Swagger documentation
- Apache Kafka for message streaming between collection and processing layers
- Apache Spark Structured Streaming for continuous data processing
- PostgreSQL for structured analytics data and MongoDB for raw document storage
- Dash and Plotly for the live interactive dashboard
- Apache Airflow for pipeline scheduling and orchestration
- NLTK with VADER for sentiment analysis
- scikit-learn for TF-IDF vectorisation and K-Means clustering
- Docker and Docker Compose for containerisation
- Kubernetes for cloud deployment with autoscaling
- GitHub Actions for CI/CD automation

Project 2: Healthcare Data Engineering Platform (Healthstream)

Healthstream is a production-style data engineering platform that simulates how large healthcare companies process medical insurance claims every day. Instead of using real patient data, which is protected by privacy laws, the system generates realistic synthetic data using Python and processes it through a full enterprise-grade technology stack. The project covers real-time streaming, batch processing, data warehousing, REST API development, dashboard visualisation, and machine learning for fraud detection, all running together as one system.

The Problem It Solves

In healthcare, insurance companies receive millions of claims every day from hospitals and doctors. Each claim says a patient received a certain treatment and the insurance company should pay a certain amount. Processing these claims involves validating the data, detecting fraud, storing records in a warehouse, running daily reports, and showing analytics to business teams. Healthstream builds all of those pieces as one working system.

Data Generation and Streaming

A Python script using the Faker library creates realistic patient records, hospital records, and insurance claims. Each claim includes a patient ID, hospital ID, treatment code, diagnosis code using real ICD-10 medical codes, a claim amount, an approved amount, insurance status, a timestamp, and a fraud score. About 5 percent of claims are intentionally made fraudulent by inflating the claim amount to 3 to 10 times the normal price. The generator runs continuously and sends 30 new claims every 3 seconds.

These claims go into Apache Kafka, which acts as a message broker. A consumer service reads every message, checks that all required fields are present and amounts are within valid range, then routes each claim: those with a fraud score of 0.7 or higher go to the fraud-alerts topic, and clean claims go to the validated-claims topic. This routing happens in real time within milliseconds of a claim arriving.

Stream Processing and Data Warehouse

Apache Spark reads the raw-claims stream continuously, parses the JSON, drops records with missing fields or invalid amounts, adds calculated columns like approval ratio and a risk flag, and writes the cleaned data to PostgreSQL every 10 seconds. PostgreSQL serves as the data warehouse with tables for patients, hospitals, claims, treatments, fraud alerts, and a daily analytics summary. The schema uses proper foreign keys and indexes so queries run fast even with large data volumes. The database is seeded on first startup with 30 days of historical claims so the dashboard has data to show immediately.

Batch Processing and Scheduling

Apache Airflow runs four scheduled jobs. The daily ETL runs every morning and calculates totals for the previous day, updates patient risk scores based on recent claim history, and updates hospital performance scores based on fraud and denial rates. The batch processing job runs every 6 hours and applies rule-based fraud detection, flagging claims that are more than 3 times the average cost for their treatment type and flagging patients who submitted

more than 5 claims in 24 hours. A weekly reporting job runs every Monday, and a cleanup job runs every Sunday to remove old resolved alerts.

A scikit-learn Random Forest classifier adds a machine learning layer, predicting the probability that a claim is fraudulent based on features like claim amount, approval ratio, insurance type, and hospital type.

API and Dashboard

A FastAPI backend exposes the stored data through REST endpoints for fetching the latest claims, getting daily cost trends, viewing fraud alerts, finding high-risk patients, and comparing hospital performance. The API has automatic interactive documentation at the /docs URL where every endpoint can be tested in the browser.

The Streamlit dashboard has six pages: an overview with total claims, fraud rate, and a 30-day cost trend; a cost trends page; a fraud detection page with a filterable alerts table; a hospital performance ranking; a patient risk page; and a live claims feed where fraudulent records are highlighted in red. The dashboard auto-refreshes every 30 seconds so new data can be watched arriving in real time.

Deployment

The entire stack runs locally with a single command using Docker Compose, containerising all eleven services with health checks and startup dependencies. Kubernetes deployment manifests are included for running the system on a cloud cluster, along with a comprehensive test suite covering unit tests for the data generator and ML models, API schema tests, and end-to-end integration tests.

Skills Gained

Before this internship, I could write Python scripts and had a basic understanding of SQL. By the end, I can design and build distributed data pipelines, work with five different types of databases, deploy containerised applications, write APIs, automate workflows with Airflow, and deploy to Kubernetes.

Technical Skills

- PySpark ETL pipelines and MLlib models
- Kafka producers and consumers
- Elasticsearch indexes with custom mappings
- Graph data modelling in Neo4j with Cypher
- GitHub Actions CI/CD pipelines
- Multi-service containerisation with Docker Compose
- Container orchestration with Kubernetes

Beyond the technical skills, I learned how to read documentation and figure out new tools. Every week introduced something new, and the only way to get through it was to read, try, fail, debug, and try again. That process, more than any individual technology, is what improved my problem-solving ability.

Problem-Solving Growth

At the start of the internship, when something did not work, my first instinct was to search for the exact error message and copy a solution. By the end, my approach had changed. I now read the error, think about what the system is doing, form a hypothesis about what is wrong, and test it. That shift from reactive to analytical debugging is the clearest sign of growth.

The big data week was where this was tested most. Hadoop, Hive, Spark, and HBase each have their own configuration requirements, and getting them to work together involved debugging errors that were not always obvious. I learned to read stack traces carefully, check logs in the right places, and isolate which component was causing the problem.

The capstone project was the ultimate test because it had no step-by-step instructions. I had to design the architecture, decide which tools to use for each part, and figure out how to connect them. When the Spark streaming job could not connect to Kafka, I had to understand the network configuration between Docker containers. When the Airflow DAG failed, I had to read the task logs to find which step broke and why.

Conclusion

This internship covered more ground than I expected when I started. The curriculum was designed to build skills progressively, starting with Python fundamentals, moving through databases and data science, then into big data and real-time processing, and finally into deployment and orchestration. Each week built on the previous one, and the capstone project required everything at once.

The most important thing I take away from this experience is not any single technology, but the confidence to pick up a new tool, understand how it fits into a larger system, and build something real with it. Data engineering is fundamentally about connecting systems together, and that is exactly what this curriculum taught me to do.